

pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi¹, Arnaud Delorme^{1,2}, and Scott Makeig¹

¹ Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA ² Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Independent Component Analysis (ICA) separates electroencephalographic and magnetoencephalographic (EEG/MEG) recordings into maximally independent sources that isolate brain, muscle, and artifact activities for downstream analysis (Iversen & Makeig, 2019; Makeig et al., 1995; Vigário et al., 1997). Adaptive Mixture ICA (AMICA) (Palmer et al., 2006, 2007, 2012) models each source with a flexible, self-adjusting probability density and lets several ICA models coexist, one per segment of a recording. Among the algorithms benchmarked by Delorme et al. (2012) it returned the least dependent and most dipolar decompositions, properties that study links to physiological interpretability. Its reference implementation, by Jason Palmer, is a Fortran program distributed as a compiled binary and usually run from MATLAB/EEGLAB; it runs only on the central processing unit (CPU) and has no Python interface.

pamica reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA graphics processing units (GPUs, via CUDA), and Apple GPUs (through Apple's MLX array framework (Hannun et al., 2023)). It reimplements the algorithm on PyTorch (Paszke et al., 2019), NumPy (Harris et al., 2020), and SciPy (Virtanen et al., 2020), and exposes a scikit-learn-style estimator under a BSD-3-Clause license. It writes the format EEGLAB's AMICA loader reads, so established MATLAB tooling consumes its results unchanged. The software is at <https://github.com/scen/pAMICA> (archived at doi:10.5281/zenodo.21312148).

Statement of need

AMICA decompositions are well suited to equivalent-dipole source localization and automated component classification (Pion-Tonachini et al., 2019). Yet the reference implementation is used mainly through MATLAB, a growing obstacle as neuroimaging analysis moves toward Python, for example MNE-Python (Gramfort et al., 2013): modern pipelines need an AMICA that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically.

General-purpose Python ICA implementations do not fill this gap. scikit-learn and MNE-Python provide FastICA (Hyvärinen & Oja, 2000) and Infomax (Bell & Sejnowski, 1995; Lee et al., 1999), while Picard (Ablin et al., 2018) offers faster-converging maximum-likelihood ICA; none implement AMICA's mixture of models, adaptive generalized-Gaussian densities, or Newton updates, so none can reproduce its decompositions. pamica is for analysts who want AMICA-quality decompositions in Python, for anyone with a GPU who wants faster runs than the CPU-only binary, and for methodologists who need a transparent reference to build on.

41 The parity measurements reported here use EEG. The algorithm itself is modality-agnostic,
42 and an external user has fit rank-reduced, Maxwell-filtered MEG with it end to end, but parity
43 with the reference has not yet been measured on MEG.

44 State of the field

45 pamica is designed to work alongside the reference Fortran AMICA used with EEGLAB (Delorme
46 & Makeig, 2004): it keeps the same output format, adds a Python API with GPU support, and
47 can run the reference Fortran itself through a bundled dependency-free native build. Three
48 other Python AMICA reimplementations have appeared as of August 2026 (Esmaili, 2025;
49 Herforth, 2026; Huberty, 2025), oriented toward MNE-Python; pamica adds a scikit-learn-style
50 array API, output in EEGLAB's exact on-disk layout, an MLX backend, and an optional MNE-
51 Python wrapper. Its validation includes score functions checked against the literal Fortran
52 expressions to floating-point resolution, and a distributional comparison for the non-identifiable
53 multi-model case.

54 Software design

55 The central design decision was to define correctness as numerical parity with Palmer's Fortran,
56 and most of the other choices follow from it. Wrapping the binary would have secured parity but
57 inherited its CPU-only, MATLAB-facing design, while a reimplementations must demonstrate
58 parity. pamica ports the algorithm natively and also ships the reference binary as a runnable
59 engine, so users can check the parity claims on their own data and hardware. For the same
60 reason the port follows the reference's natural-gradient (Amari, 1998) expectation-maximization
61 (EM) formulation. An early Adam/autograd backend was removed because it converged to
62 different optima, which made results labeled "AMICA" hard to compare with the reference. The
63 port covers exact-EM mixture updates, a positive-definite Newton step (Palmer et al., 2008),
64 symmetric sphering, the five source-density families, a mixture of ICA models, component
65 sharing, and the mutual-information metrics used to score separation quality (Frank et al.,
66 2023).

67 Three array backends (PyTorch, MLX, NumPy) implement the same algorithm behind a
68 common estimator API, with the reference Fortran a fourth. NumPy is kept as a readable
69 executable specification. MLX is included because it is the more efficient path on Apple GPUs:
70 in the block-size sweep, PyTorch's Metal backend was slower than MLX at every size, and at
71 pamica's shipped defaults slower than the CPU it runs beside (see the documentation). Double
72 precision is the default because the parity comparisons need it; single precision is available for
73 Apple GPUs, which have no float64.

74 Validation

75 Parity is measured two ways: by Hungarian-matched component correlation, and by the Amari
76 distance (Amari et al., 1996), a relabeling- and scale-invariant metric that needs no assignment
77 step. Both implementations ran AMICA's default 2000 iterations with Newton disabled
78 (pamica's own default), to isolate the algorithm from initialization. With Newton enabled,
79 some of the weakest components settle into different basins of equal or higher likelihood
80 from different starts, in the reference's runs as in pamica's: against one reference run, one
81 pamica seed of three differed on eight of seventy components and the other two matched
82 at 0.995 and 0.996, and two reference runs from different seeds differed on two. From a
83 shared start the two implementations end with the same components (correlation 0.9999998).
84 The single-model comparison uses a well-determined external recording (NEMAR on002718,
85 $k \approx 153$, where k = frames over squared channel count (Frank et al., 2025)) alongside
86 the bundled 32-channel sample ($k \approx 30$). On the bundled sample one of the five reference

87 runs ended in a lower-likelihood basin and sets the five-pair Amari mean; over 50 run pairs
 88 the distance between the implementations (0.004) is close to the reference's own run-to-run
 89 distance (0.005). A mixture of ICA models is not partition-identifiable, so the implementations
 90 are compared by the distributions of solutions they sample, over ensembles of 20 runs each
 91 (Figure 1). One-sided run-level permutation tests of whether cross-implementation agreement
 92 is worse than the reference's own run-to-run agreement give $p = 0.88$ by correlation and
 93 $p = 0.051$ by Amari distance, by which pamica's ensemble spreads slightly more than the
 94 reference's.

Table 1: Parity of pamica with the Fortran reference. Multi-model rows are over 20-run ensembles (190 within-, 400 cross-implementation pairs); sd is the standard deviation, given where computed; p values are one-sided run-level permutation tests.

Regime	Metric (dataset)	Result (mean)
Single	Log-likelihood gap (on002718)	within ~ 0.0005 of -3.6993
Single	Component correlation (on002718)	0.998
Single	Amari distance (bundled)	0.011 (5 run pairs); 0.004 (50 pairs)
Single	Score functions, sufficient statistics	round-off, $\leq 2 \times 10^{-15}$ (relative)
Multi	Correlation, one run: cross; within-Fortran	0.632; 0.626 (sd 0.04)
Multi	Amari, one run: cross; within-Fortran	0.172; 0.166 (sd 0.02)
Multi	Ensemble agreement, cross — within-Fortran	correlation +0.006 ($p = 0.88$); Amari +0.005 ($p = 0.051$)
Multi	Ensemble log-likelihood: Fortran; pamica	-3.3543 ; -3.3541 (Kolmogorov-Smirnov $p = 0.83$)

Multi-model AMICA (n_models=2): pamica vs Fortran ensembles, real sample EEG

Dashed vertical lines mark each distribution's mean.

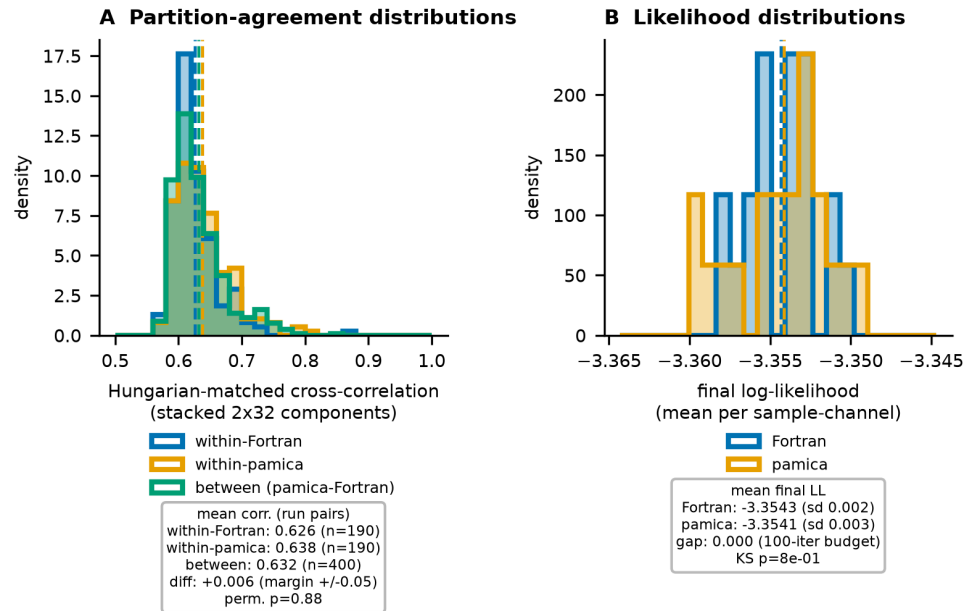


Figure 1: Multi-model ensemble partition-correlation (A) and log-likelihood (B) distributions, 20 pamica and 20 Fortran fits of the sample EEG; dashed lines mark each mean. A's three distributions overlap: the single-run correlation of ~ 0.63 matches the reference's agreement with itself. B's two means lie 0.0002 apart on a ~ 0.02 axis.

From a shared start with the harness settings, the single-precision MLX backend and the double-precision PyTorch backend differ by 5×10^{-6} in log-likelihood after 100 iterations, with mean matched component correlation 0.9999999 and Amari distance 5×10^{-5} . In the throughput benchmark's runs on a 30,000-frame excerpt of the external recording, the backends that share a start agree after 25 iterations to 10^{-5} in log-likelihood on 32 and 48 channels; with 70 channels ($k \approx 6$) they differ by up to 10^{-3} , about the effect of perturbing one sample by 10^{-9} μ V. Double precision remains the default for parity work, and double-precision CUDA is the reproducible NVIDIA path.

On real 70-channel EEG at block_size=512, per-iteration cost is 25 ms for MLX on an Apple GPU, 39 ms for double-precision CUDA on an RTX 4090, and 30 ms for native Fortran on a 24-core i9-13900K, against 193 ms for PyTorch on an Apple-Silicon CPU and 255 ms for PyTorch-Metal. That comparison is block-size-confounded: PyTorch-Metal is far more block-size-sensitive than the CPU it runs beside, falling from 431 to 30.5 ms/iteration on the bundled 32-channel sample between block_size=512 and pamica's current 8192 default, still behind the CPU's 21.7 ms/iteration there, and to 13.5 ms/iteration, below the CPU's 15.8 ms, at a further-tuned single-block setting that is memory-limited rather than a free win, since peak block memory scales with block_size, which is why 8192 stays the shipped default. MLX stays the fastest Apple backend throughout the sweep. Full tables, the data-size sweep, and reproduction commands are in the [documentation](#); the correctness harness never uses synthetic data. The harness and sample data are in the source repository, and every row of Table 1 but the two on002718 entries re-runs from the bundled sample alone.

Research impact statement

AMICA's source model and estimation were published in 2006-2008 (Palmer et al., 2006, 2007, 2008), and it ranked first of the twenty-two algorithms benchmarked by Delorme et al. (2012). MNE-Python is publicly weighing which of the Python ports to adopt (mne-tools/mne-python issue 13819), where an MNE core developer and the author of a competing port lean toward pamica, since this Center develops AMICA and maintains the reference Fortran and EEGLAB. sccn/pAMICA has been public since 2021, with public issues, pull requests, nine releases (six on PyPI), and a Zenodo archive. Three researchers outside the author group have filed issues, two mid-analysis on their own data: a 236-channel, 8.3-million-sample EEG decomposition (issue 207) and Maxwell-filtered 306-channel MEG (issue 221); the third is adopting AMICA for a multiverse analysis in MNE-Python (issue 206). Integration into this Center's Python preprocessing and into NEMAR (Delorme et al., 2022), the public archive we operate (800 datasets, ~40,000 participants, 55 TB), is in progress.

AI usage disclosure

Generative AI was used in this project, disclosed here under the journal's policy.

Tools. Anthropic's Claude models (Sonnet and Opus families), through the Claude Code command-line assistant. The instructions given to them are public in the repository (AGENTS.md, CLAUDE.md, .rules/).

Scope. The source code (translating the reference Fortran into Python, refactoring, scaffolding tests), the documentation, and the drafting and copy-editing of this manuscript.

Human oversight. The authors made the design decisions and take responsibility for the result. Parity as the correctness criterion, the natural-gradient EM formulation, the backend architecture, the distributional treatment of the multi-model case, and the acceptance thresholds were chosen by the authors. Every AI-assisted change was reviewed by a human before merge and run through the parity harness, which scores output against the reference binary on real recordings. The reported numbers came from running the software and were checked against their run records, as was every bibliographic entry.

Acknowledgments

We thank Jason Palmer and his advisor Ken Kreutz-Delgado, co-developers of AMICA, for the reference implementation, and the EEGLAB community for the tools and sample data used to validate this work. Two authors developed the methods pamica builds on: S.M. co-developed AMICA (Palmer et al., 2012) and A.D. is a lead developer of EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of Health grant R01-NS047293 (to A.D. and S.M.).

References

- Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*, 66(15), 4040–4053. <https://doi.org/10.1109/TSP.2018.2844203>
- Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276. <https://doi.org/10.1162/089976698300017746>
- Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal separation. *Advances in Neural Information Processing Systems*, 8, 757–763.

- 159 Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind
160 separation and blind deconvolution. *Neural Computation*, 7(6), 1129–1159. [https://doi.
161 org/10.1162/neco.1995.7.6.1129](https://doi.org/10.1162/neco.1995.7.6.1129)
- 162 Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-
163 trial EEG dynamics including independent component analysis. *Journal of Neuroscience
164 Methods*, 134(1), 9–21. <https://doi.org/10.1016/j.jneumeth.2003.10.009>
- 165 Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG
166 sources are dipolar. *PLoS ONE*, 7(2), e30135. [https://doi.org/10.1371/journal.pone.
167 0030135](https://doi.org/10.1371/journal.pone.0030135)
- 168 Delorme, A., Truong, D., Youn, C., Sivagnanam, S., Stirn, K., Yoshimoto, K., Poldrack,
169 R. A., Majumdar, A., & Makeig, S. (2022). NEMAR: An open access data, tools and
170 compute resource operating on neuroelectromagnetic data. *Database*, 2022, baac096.
171 <https://doi.org/10.1093/database/baac096>
- 172 Esmaeili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and
173 scientific EEG workflows*. <https://github.com/snesmaeili/amica>.
- 174 Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2023).
175 An exploration of optimal parameters for efficient blind source separation of EEG
176 recordings using AMICA. *2023 IEEE 23rd International Conference on Bioinformatics and
177 Bioengineering (BIBE)*, 205–210. <https://doi.org/10.1109/BIBE60311.2023.00040>
- 178 Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2025).
179 Quantitative exploration of sufficient data quantities for EEG decomposition using AMICA.
180 *2025 12th International IEEE/EMBS Conference on Neural Engineering (NER)*, 532–536.
181 <https://doi.org/10.1109/NER61569.2025.11588993>
- 182 Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj,
183 R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data
184 analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. [https://doi.org/10.3389/
185 fnins.2013.00267](https://doi.org/10.3389/fnins.2013.00267)
- 186 Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible
187 machine learning on apple silicon*. <https://github.com/ml-explore/mlx>.
- 188 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
189 Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with
190 NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 191 Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component
192 analysis*. <https://github.com/DerAndereJohannes/pyamica>.
- 193 Huberty, S. (2025). *Amica-python: Python implementation of adaptive mixture ICA*. [https:
194 //github.com/scott-huberty/amica-python](https://github.com/scott-huberty/amica-python).
- 195 Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications.
196 *Neural Networks*, 13(4-5), 411–430. [https://doi.org/10.1016/S0893-6080\(00\)00026-5](https://doi.org/10.1016/S0893-6080(00)00026-5)
- 197 Iversen, J. R., & Makeig, S. (2019). MEG/EEG data analysis using EEGLAB. In
198 *Magnetoencephalography: From signals to dynamic cortical networks* (pp. 391–406).
199 Springer International Publishing. https://doi.org/10.1007/978-3-319-62657-4_8-1
- 200 Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using
201 an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural
202 Computation*, 11(2), 417–441. <https://doi.org/10.1162/089976699300016719>
- 203 Makeig, S., Bell, A. J., Jung, T.-P., & Sejnowski, T. J. (1995). Independent component
204 analysis of electroencephalographic data. *Advances in Neural Information Processing
205 Systems*, 8, 145–151.

- 206 Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2006). Super-Gaussian mixture source model
207 for ICA. *Independent Component Analysis and Blind Signal Separation: Proceedings of the*
208 *6th International Conference (ICA 2006)*, 854–861. https://doi.org/10.1007/11679363_106
- 209 Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). *AMICA: An adaptive*
210 *mixture of independent component analyzers with shared components*. Swartz
211 Center for Computational Neuroscience, University of California San Diego.
212 https://sccn.ucsd.edu/~jason/amica_a.pdf
- 213 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2007). Modeling and estimation
214 of dependent subspaces with non-radially symmetric and skewed densities. In M. E. Davies,
215 C. J. James, S. A. Abdallah, & M. D. Plumbley (Eds.), *Independent component analysis*
216 *and signal separation: Proceedings of the 7th international conference (ICA 2007)* (pp.
217 97–104). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-74494-8_13
- 218 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the
219 ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal*
220 *Processing (ICASSP)*, 1805–1808. <https://doi.org/10.1109/ICASSP.2008.4517982>
- 221 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
222 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-
223 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,
224 8024–8035.
- 225 Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated
226 electroencephalographic independent component classifier, dataset, and website.
227 *NeuroImage*, 198, 181–197. <https://doi.org/10.1016/j.neuroimage.2019.05.026>
- 228 Vigário, R., Jousmäki, V., Härmäläinen, M., Hari, R., & Oja, E. (1997). Independent component
229 analysis for identification of artifacts in magnetoencephalographic recordings. *Advances in*
230 *Neural Information Processing Systems*, 10, 229–235.
- 231 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
232 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
233 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
234 261–272. <https://doi.org/10.1038/s41592-019-0686-2>